



AUBURN

UNIVERSITY

CPU Scheduling

Overview

Objectives

- Review Process Scheduling.
- Learn, evaluate, and analyze **First-Come First-Served (FCFS)**.
- Learn, evaluate, and analyze **Shortest Job First (SJF)**.
- Learn, evaluate, and analyze **Shortest Remaining Time First (SRTF)**.
- Learn, evaluate, and analyze **Round Robin (RR)**.
- Learn, evaluate, and analyze **Multi-Level Feedback Queue (MLFQ)**.

Requirements (CPU Scheduling)

- Read Section Basic Concepts
- Read Section Scheduling Criteria
- Read Section Scheduling Algorithms



The Big Picture

- What is the **road map** to study this topic?



The Big Picture

? **What** is **CPU Scheduling**?

? CPU Scheduling is the **sharing** of the CPU among **ready** processes

? **Sharing** always raises these questions

? How does a process **behave**?

? Do all processes **have the same needs/behavior**?

? What is the **objective** of your sharing/scheduling strategy/policy?

? In other words,

? how will you **evaluate** your sharing/scheduling strategy/policy?

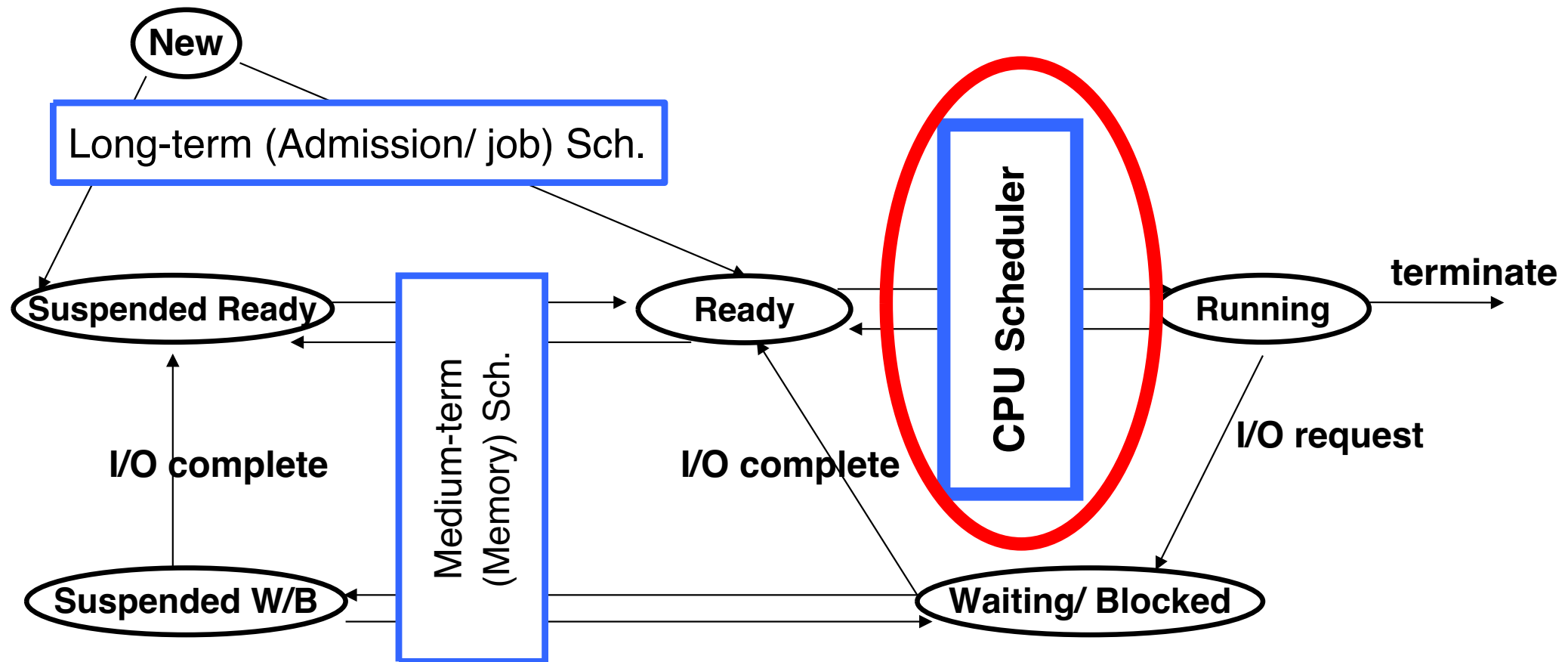
? What are the **metrics** of your sharing/scheduling strategy/policy?

? **What** are the different **CPU scheduling strategies**?

CPU Scheduler?

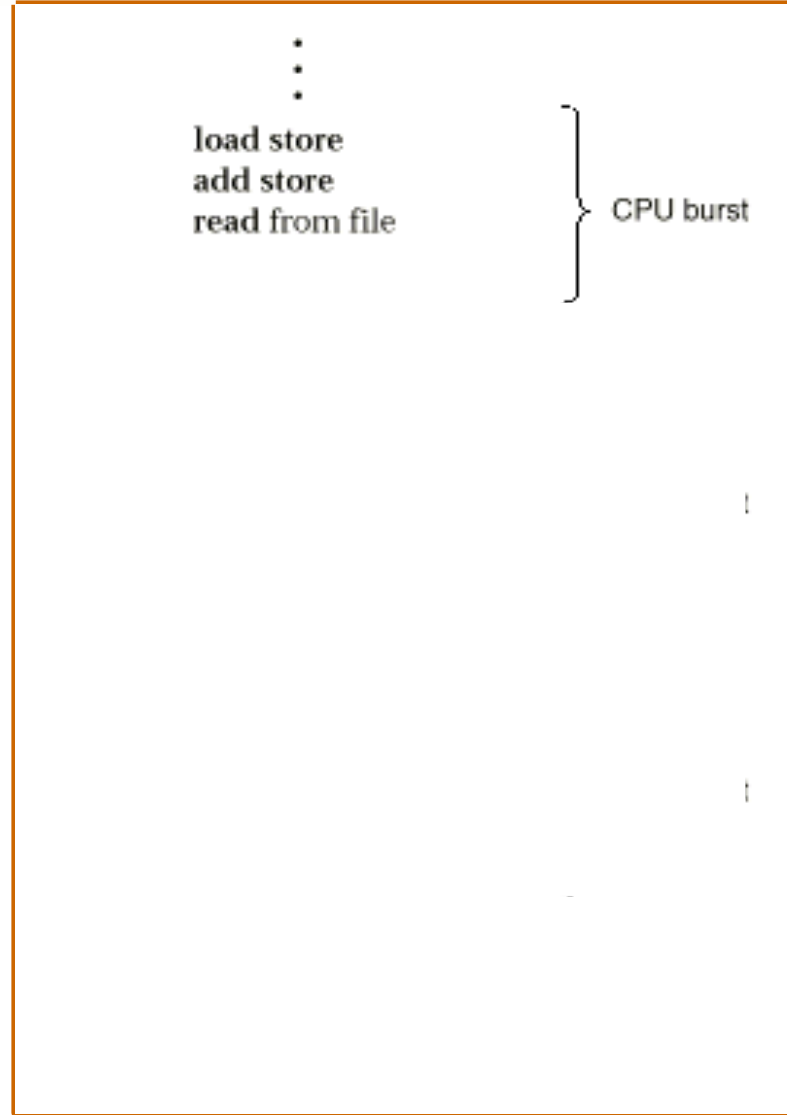
- **What** is CPU Scheduling?
 - How does a process **behave** in general?
 - Do all processes **have the same needs**?
 - Answer: No!
 - **CPU-Bound** processes
 - **I/O-Bound** processes

Expanded State Transition Diagram



How does a process behave?

Answer: Alternating Sequence of CPU And I/O Bursts



Types of Processes

? Bursts of CPU usage alternate with periods of I/O wait

- a) a **CPU-bound** process
- b) an **I/O bound** process

What is the Objective of the CPU Scheduling? (Cont'd)

? Answer

? We must first define the **metrics** to evaluate a CPU scheduling strategy.

? Metrics to evaluate CPU Scheduling strategies

↑ **CPU Utilization**: fraction of time a CPU is used by **user** processes

↑ **Throughput**: N/T

where N = Number of Processes Completed and T = Observation time

↓ **Response Time**: $T_{\text{FirstTime}} - T_a$

where $T_{\text{FirstTime}}$ = Time a process gets CPU the first time and T_a = Process Arrival time

↓ **Turnaround Time**: $T_{\text{complete}} - T_a$

where T_{complete} = Time a process completes and terminates

↓ **Waiting Time**: T_{waiting}

where T_{waiting} = Total time spent by a process in the **ready** queue

Objectives of CPU Scheduling

All systems

Batch systems

Interactive systems

Real-time systems

Objectives of CPU Scheduling

All systems

Fairness - giving each process a fair share of the CPU

Policy enforcement - seeing that stated policy is carried out

Balance - keeping all parts of the system busy

Batch systems

Throughput - maximize jobs per hour

Turnaround time - minimize time between submission and termination

CPU utilization - keep the CPU busy all the time

Interactive systems

Response time - respond to requests quickly

Proportionality - meet users' expectations

Real-time systems

Meeting deadlines - avoid losing data

Predictability - avoid quality degradation in multimedia systems

CPU Scheduling Policies

- Objectives
 - Minimize turnaround time
 - Minimize response time
 - Maximize throughput
- Preemptive and non preemptive scheduling
- CPU Scheduling Policies
 - First-Come First-Served (FCFS)
 - Shortest Job First (SJF)
 - Shortest Remaining Time First (SRTF)
 - Round Robin(RR)
 - Multi-Level Feedback Queue (MLFQ)

Preemptive vs. Non-Preemptive

? Preemptive: OS can **forcibly** take back the CPU.

? Examples:

SRTF

RR

MLFQ

? Non-Preemptive: the process **voluntarily** relinquishes the CPU.

? Examples:

FCFS

SJF

First-Come First Served (FCFS)

Policy: the process that arrives first is allocated the CPU first.

Example

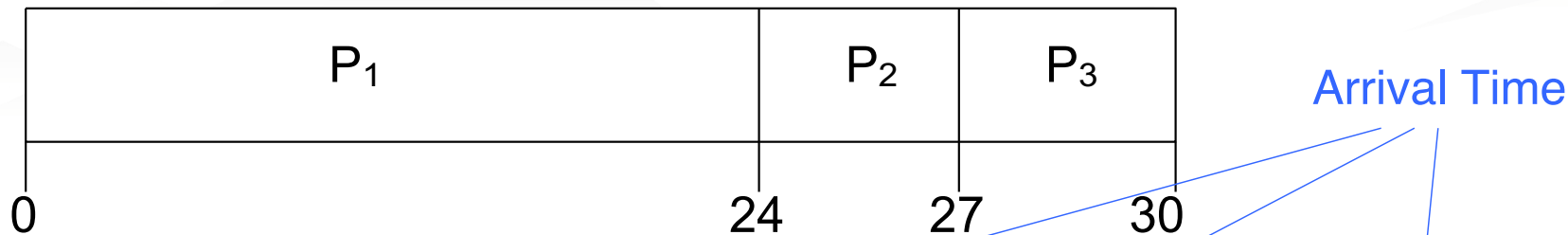
<u>Process</u>	<u>Burst Time</u>	<u>Arrival Time</u>
P_1	24	0
P_2	3	20
P_3	3	25

☐ The processes arrive in the order: P_1 , P_2 , P_3

Gant Chart (ignoring context switch time)



FCFS Response Time, Turnaround Time, Waiting Time



Response Time: (Time First Time On CPU – Arrival Time)

$$T_{r,p1} = (0 - 0) = 0; T_{r,p2} = (24 - 20) = 4; T_{r,p3} = (27 - 25) = 2$$

$$\text{Average Response Time} = \text{ART} = (0 + 4 + 2)/3 = 2$$

Turnaround time (Completion Time – Arrival Time)

$$T_{T,p1} = (24 - 0) = 24; T_{T,p2} = (27 - 20) = 7; T_{T,p3} = (30 - 25) = 5$$

$$\text{Average Turnaround Time} = \text{ATT} = (24 + 7 + 5)/3 = 12$$

Waiting time (Time spent in ready queue)

$$T_{w,p1} = 0; T_{w,p2} = 4; T_{w,p3} = 2$$

$$\text{Average Waiting Time} = \text{AWT} = (0 + 4 + 2)/3 = 2$$

Optimal CPU Scheduling Policies

- Shortest Job First (non preemptive)
- Shortest Remaining Time First (Preemptive)

SJF and SRTF

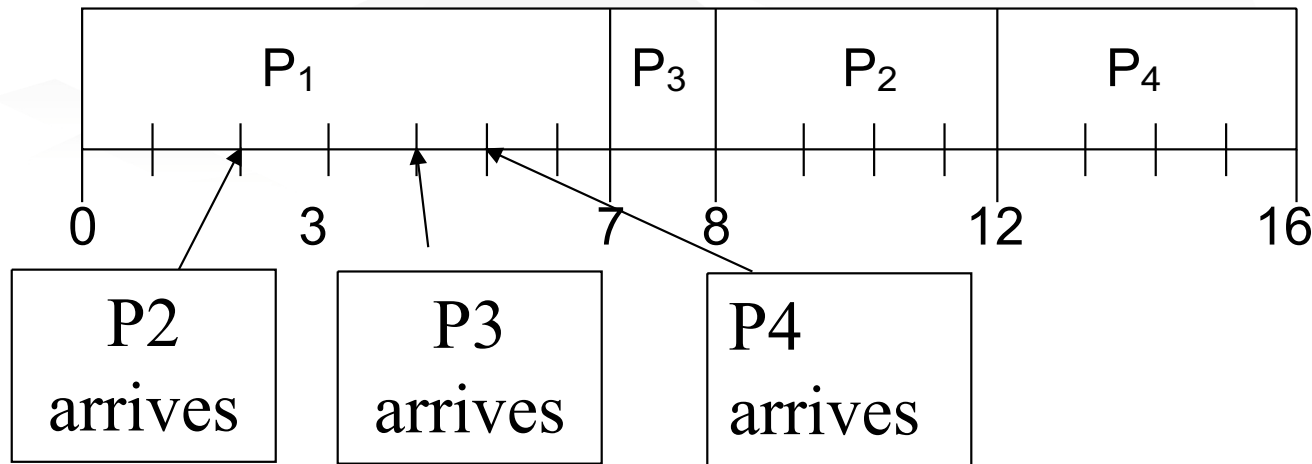
? Shortest Job First: (SJF)

- ? **non-preemptive** – once the CPU is given to a process it cannot be preempted until it completes its CPU burst
- ? Do scheduling **ONLY** when a process **finishes** and relinquishes voluntarily the CPU;
- ? **Policy**: associate with each process the length of its **next** CPU burst. Use these lengths from all processes in ready queue to schedule the process with the shortest time
- ? Fact: SJF is **optimal** – delivers minimum average turnaround time for a given set of processes

? Shortest-Remaining-Time-First (SRTF)

- ? **preemptive** – **Policy**: if a new process arrives with CPU burst length less than remaining time of current executing process, **preempt** (take back CPU)
- ? Do scheduling whenever a process **arrives** or **finishes**

SJF Example



<u>Process</u>	<u>Burst Time</u>	<u>Arrival Time</u>
P_1	0.0	7
P_2	2.0	4
P_3	4.0	1
P_4	5.0	4

Response Time: (Time First Time On CPU – Arrival Time)

$$T_{r,p1} = (0 - 0) = 0; T_{r,p2} = (8 - 2) = 6; T_{r,p3} = (7 - 4) = 3; T_{r,p4} = (12 - 5) = 7$$

$$\text{Average Response Time} = \text{ART} = (0 + 6 + 3 + 7)/4 = 16/4 = 4$$

Turnaround time (Completion Time – Arrival Time)

$$T_{T,p1} = (7 - 0) = 7; T_{T,p2} = (12 - 2) = 10; T_{T,p3} = (8 - 4) = 4; T_{T,p4} = (16 - 5) = 11$$

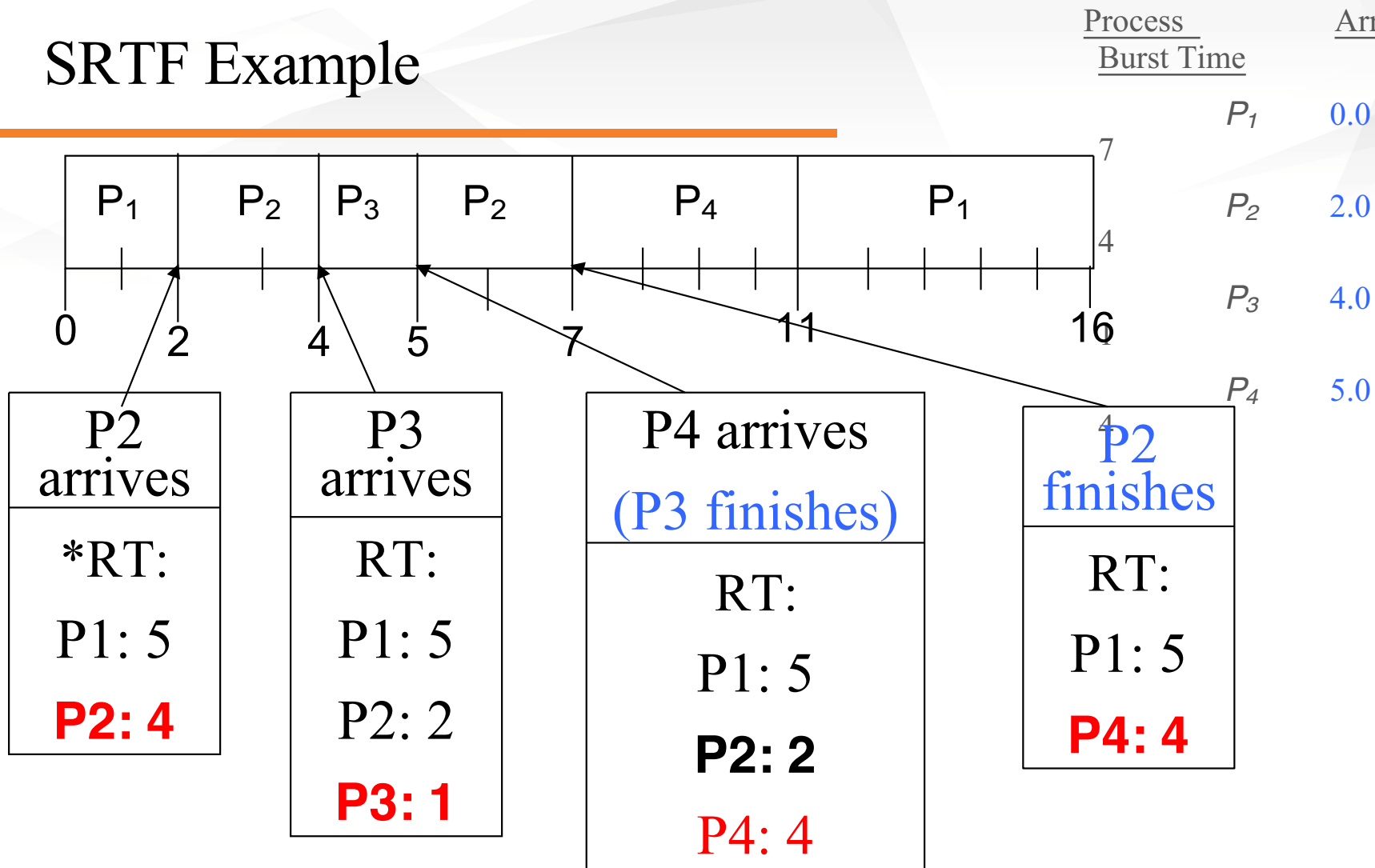
$$\text{Average Turnaround Time} = \text{ATT} = (7 + 10 + 4 + 11)/4 = 32/4 = 8$$

Waiting time (Time spent in ready queue)

$$T_{w,p1} = 0; T_{w,p2} = 6; T_{w,p3} = 3; T_{w,p4} = 7$$

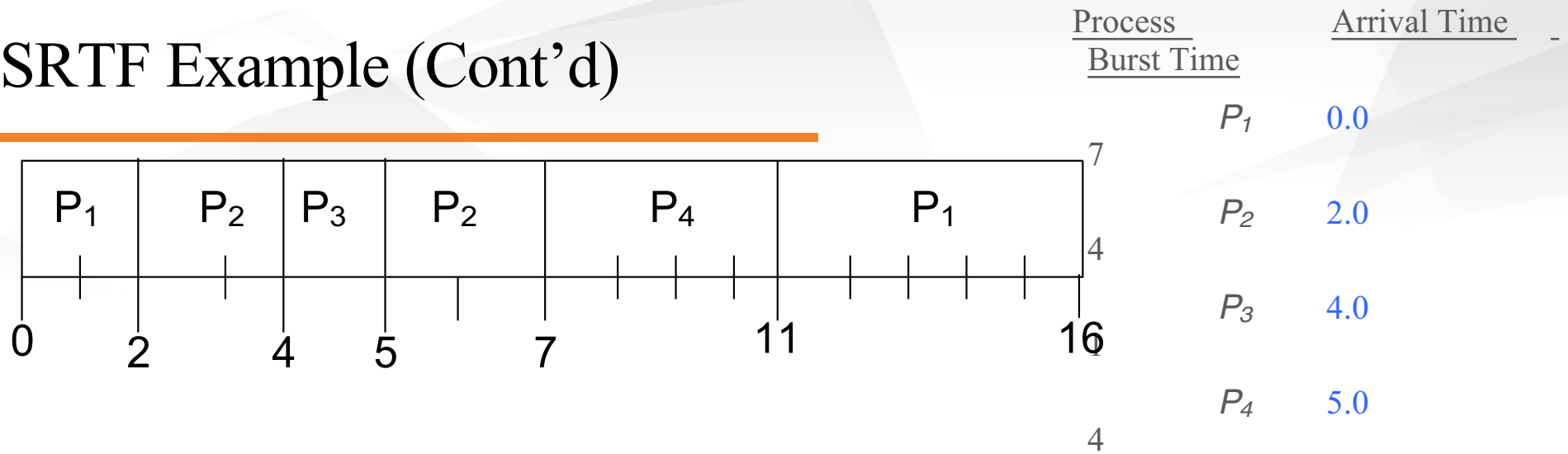
$$\text{Average Waiting Time} = \text{AWT} = (0 + 6 + 3 + 7)/4 = 16/4 = 4$$

SRTF Example



*RT: Remaining Time

SRTF Example (Cont'd)



Response Time: (Time First Time On CPU – Arrival Time)

$$T_{r,p1} = (0 - 0) = 0; T_{r,p2} = (2 - 2) = 0; T_{r,p3} = (4 - 4) = 0; T_{r,p4} = (7 - 5) = 2$$

$$\text{Average Response Time} = \text{ART} = (0 + 0 + 0 + 2)/4 = 2/4 = 0.5$$

Turnaround time (Completion Time – Arrival Time)

$$T_{T,p1} = (16 - 0) = 16; T_{T,p2} = (7 - 2) = 5; T_{T,p3} = (5 - 4) = 1; T_{T,p4} = (11 - 5) = 6$$

$$\text{Average Turnaround Time} = \text{ATT} = (16 + 5 + 1 + 6)/4 = 28/4 = 7$$

Waiting time (Time spent in ready queue)

$$T_{w,p1} = 9; T_{w,p2} = 1; T_{w,p3} = 0; T_{w,p4} = 2$$

$$\text{Average Waiting Time} = \text{AWT} = (9 + 1 + 0 + 2)/4 = 12/4 = 3$$

SJF or SRTF Are Optimal, But.....

? SJF and SRTF are in general **impossible** to implement:

? **Unable to know** (remaining) CPU time

? **Starvation problem** – Long job may never get CPU

? Can we **approximate** SRTF?

i.e, make shorter jobs serviced faster

and avoid starvation

without knowledge of remaining CPU time needed

Yes, Round Robin !

Round Robin

- Round Robin CPU Scheduling



Round Robin (RR)

❓ Policy:

1. Order processes in the ready queue in FCFS
2. Assign the CPU to the head of the list
3. CPU is assigned for a limited amount of time ❓ (**quantum**).

❓ A **quantum** usually has 10-100 milliseconds. After this time has elapsed, the CPU is **preempted** and the previously running process is added to the end of the ready queue.

❓ If there are n processes in the ready queue and the time quantum is q , then

- each process gets $1/n$ of the CPU time in chunks of at most q time units at once.
- No process waits more than $(n-1)q$ time units.

❓ Performance

- q large ❓ FIFO
- q small ❓ get better response time, but you pay a high overhead due the time spent in the context switch: q must be large with respect to context switch, otherwise overhead is too high

Example of RR with Time Quantum = 20

Process Burst Time

P₁ 53

P₂ 17

P₃ 68

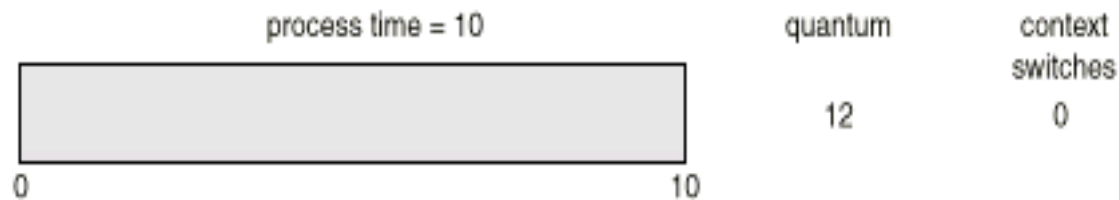
P₄ 24

? The **Gantt chart** is (ignoring the context switch time):

P ₁	P ₂	P ₃	P ₄	P ₁	P ₃	P ₄	P ₁	P ₃	P ₃	
0	20	37	57	77	97	117	121	134	154	162

? Typically, **higher average turnaround** than SJF, but **better response**

? Context switch takes time: **smaller quantum** ? **higher time overhead**



Big Picture

Relationship Between FCFS, SJF/SRTF, and RR?

? What is the problem/weakness of FCFS?

Answer: average turnaround will increase with large processes in front of smaller ones

? How does SJF address FCFS's weakness?

Answer: it favors short jobs. SJF offers OPTIMAL turnaround time

? What is SJF's weakness?

Answer: engineers are not psychics: often, we **cannot know** in advance the length of a CPU burst

? How does RR address SJF's weakness?

Answer: without knowing in advance the length of the CPU burst, by taking turns, RR insures that shorter jobs complete earlier than larger ones

? What is RR's weakness?

Answer: more context switches than SJF or FCFS ? **decreases throughput** (RR has lower throughput than FCFS)

? Can we address RR's weakness? If yes, How?

Answer: Yes, by **adapting** the quantum: increase the quantum for longer CPU bursts.

? Is there a CPU scheduling policy with adaptive quantum?

Answer: Yes, **Multi-Level Feedback Queue (MLFQ)**

Multi-Level Feedback Queue

- Multi-level Feedback Queue CPU Scheduling



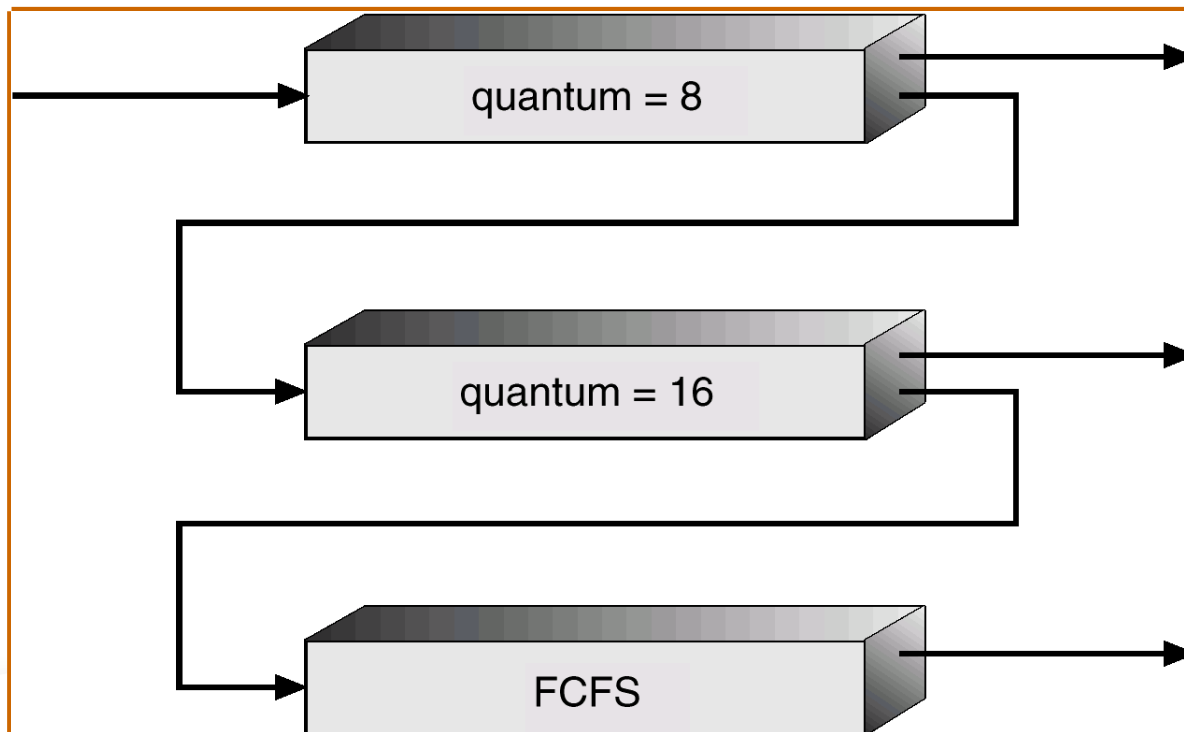
Multi Level Feedback Queue

Queues Organization:

- n queues: $Q_1, Q_2, \dots, Q_n$. Each queue Q_i has a quantum q_i : $0 < q_1 < q_2 < \dots < q_n$
- Processes are kept in FCFS order in each queue
- Processes in queue Q_1 are executed first, then $Q_2, \dots$ then Q_n .
- Upon arrival, a process is put in Q_1 . After one quantum in Q_i , it either leaves the system if done or is put in Q_{i+1} .

MLFQ Parameters

- number of queues n
- scheduling algorithm for each queue
- method used to determine when to upgrade a process (aging)



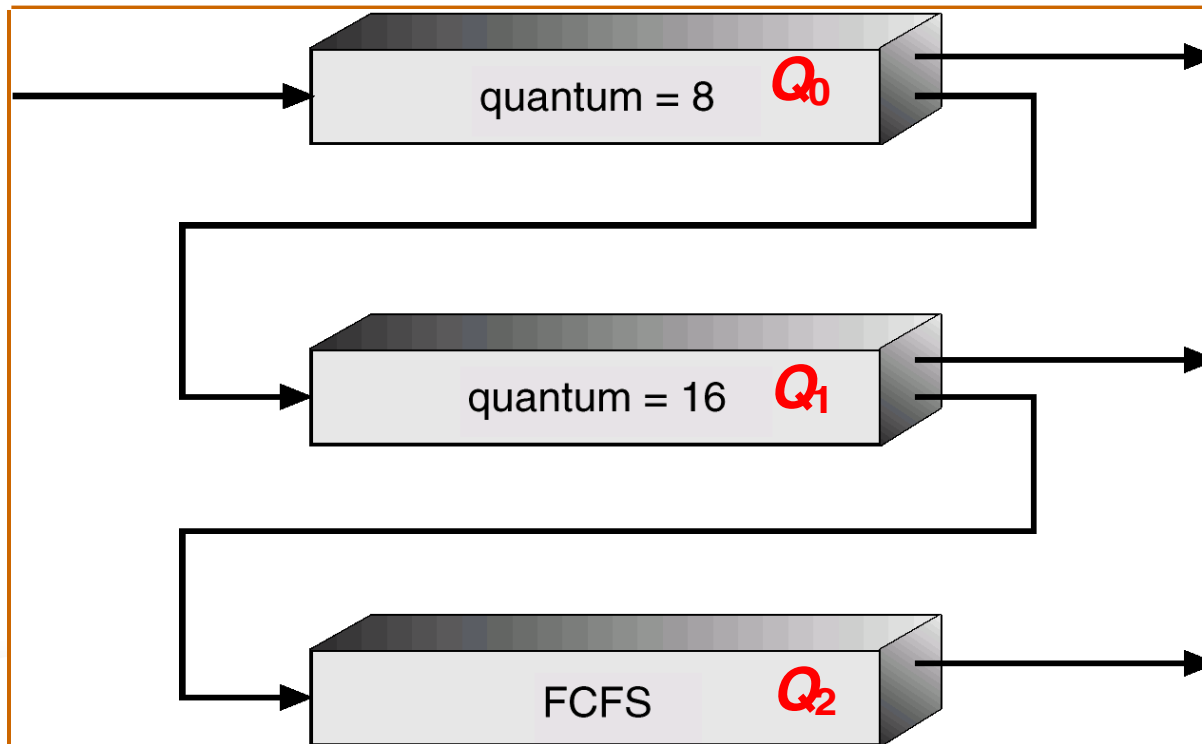
MLFQ Example

? Three queues:

- ? Q_0 – time quantum 8 milliseconds
- ? Q_1 – time quantum 16 milliseconds
- ? Q_2 – time quantum ? (FCFS)

? Scheduling

- ? MLFQ is preemptive: if a process just arrives into higher (priority) queue, it will preempt any process from lower (priority) queues.
- ? A new job enters queue Q_0 which is served FCFS. When it gains CPU, job receives 8 milliseconds. If it does not finish in 8 milliseconds, job is moved to queue Q_1 .
- ? At Q_1 job is again served FCFS and receives 16 additional milliseconds. If it still does not complete, it is preempted and moved to queue Q_2 .



MLFQ Example

Process
Burst Time

P_1

28

P_2

10

P_3

5

P_4

26

P1	P2	P3	P4	P1	P2	P4	P1	P4	
0	8	16	21	29	45	47	63	67	69

? Assumption: all processes arrive at time 0:

? In Q_0 (Quantum = 8):

P1 uses 8 and needs 20 more, ? Q_1

P2 uses 8 and needs 2 more, ? Q_1

P3 uses 5 and terminates

P4 uses 8 and needs 18 more, ? Q_1

? In Q_1 (Quantum = 16):

? P1 (20) uses 16 and needs 4 more, ? Q_2

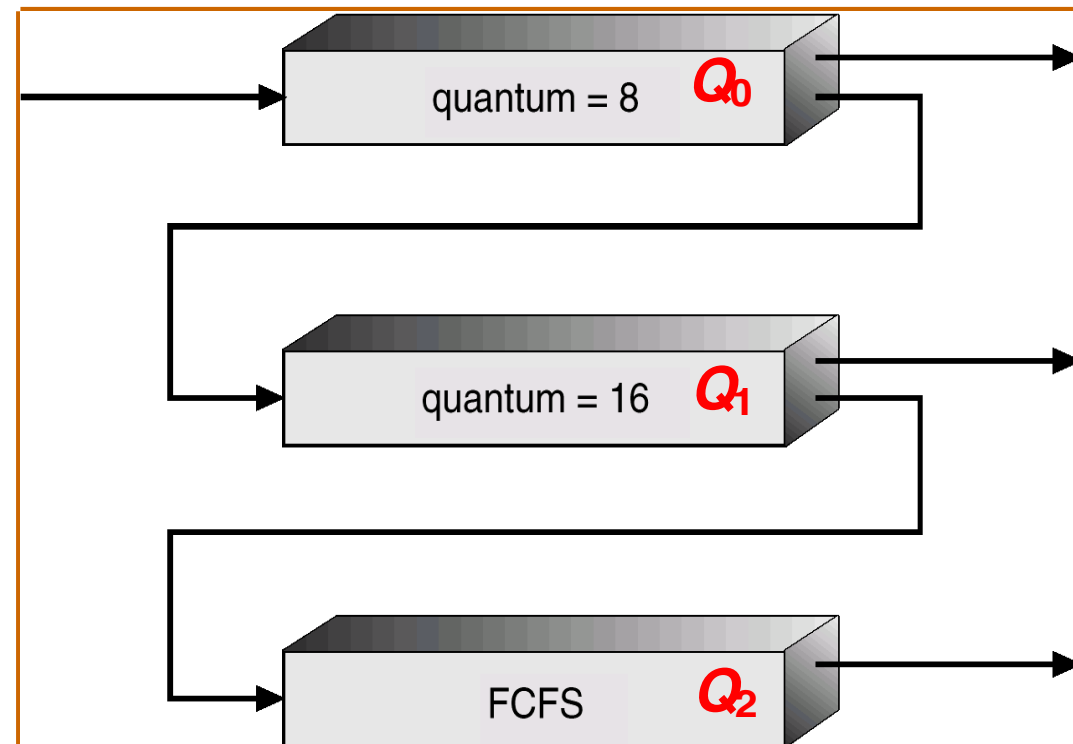
? P2 (2) uses 2 and terminates

? P4 (18) uses 16 and needs 2 more, ? Q_2

? In Q_2 (Quantum = infinity):

? P1 (4) uses 4 and terminates

? P4 (2) uses 2 and terminates

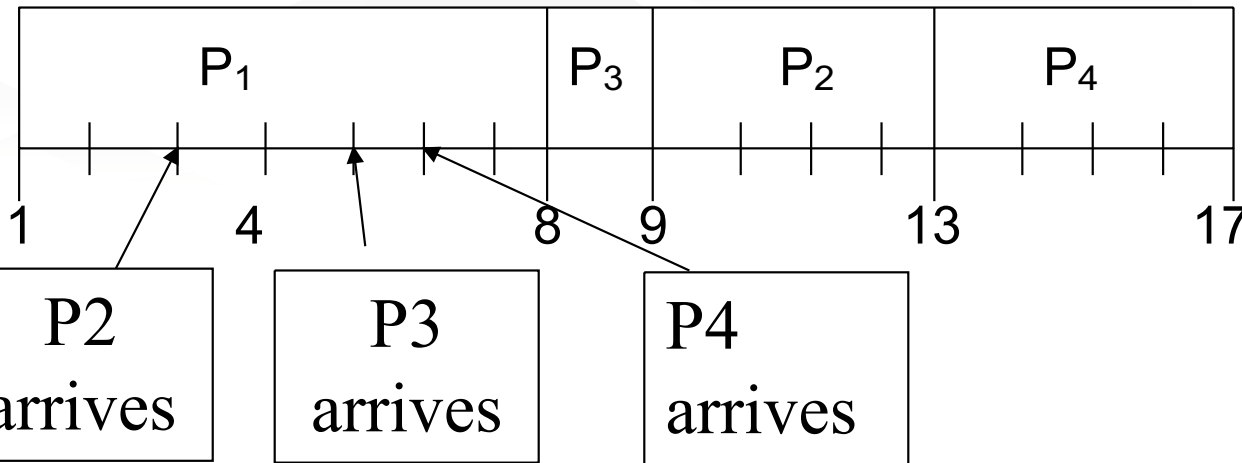


Wrap Up

- ? Review *Process Scheduling* and understand the specific role of the **CPU Scheduler**
- ? Learn, evaluate, and analyze **First-Come First-Served (FCFS)**.
- ? Learn, evaluate, and analyze **Shortest Job First (SJF)**.
- ? Learn, evaluate, and analyze **Shortest Remaining Time First (SRTF)**.
- ? Learn, evaluate, and analyze **Round Robin (RR)**.
- ? Learn, evaluate, and analyze **Multi-Level Feedback Queue (MLFQ)**.
- ? Insure that you know :
 1. the strengths and weaknesses of each CPU scheduling policy, and
 2. how these CPU scheduling policies address each other weaknesses if they do.
- ? A question: based on all what we saw, which processes should be favored? CPU-bound or I/O-bound processes?

SJF Example

Process	Burst Time	Arrival Time
P_1	1.0	7
P_2	3.0	4
P_3	5.0	1
P_4	6.0	4



Response Time: (Time First Time On CPU – Arrival Time)

$$T_{r,p1} = (1 - 1) = 0; T_{r,p2} = (9 - 3) = 6; T_{r,p3} = (8 - 5) = 3; T_{r,p4} = (13 - 6) = 7$$

$$\text{Average Response Time} = \text{ART} = (0 + 6 + 3 + 7)/4 = 16/4 = 4$$

Turnaround time (Completion Time – Arrival Time)

$$T_{T,p1} = (8 - 1) = 7; T_{T,p2} = (13 - 3) = 10; T_{T,p3} = (9 - 5) = 4; T_{T,p4} = (17 - 6) = 11$$

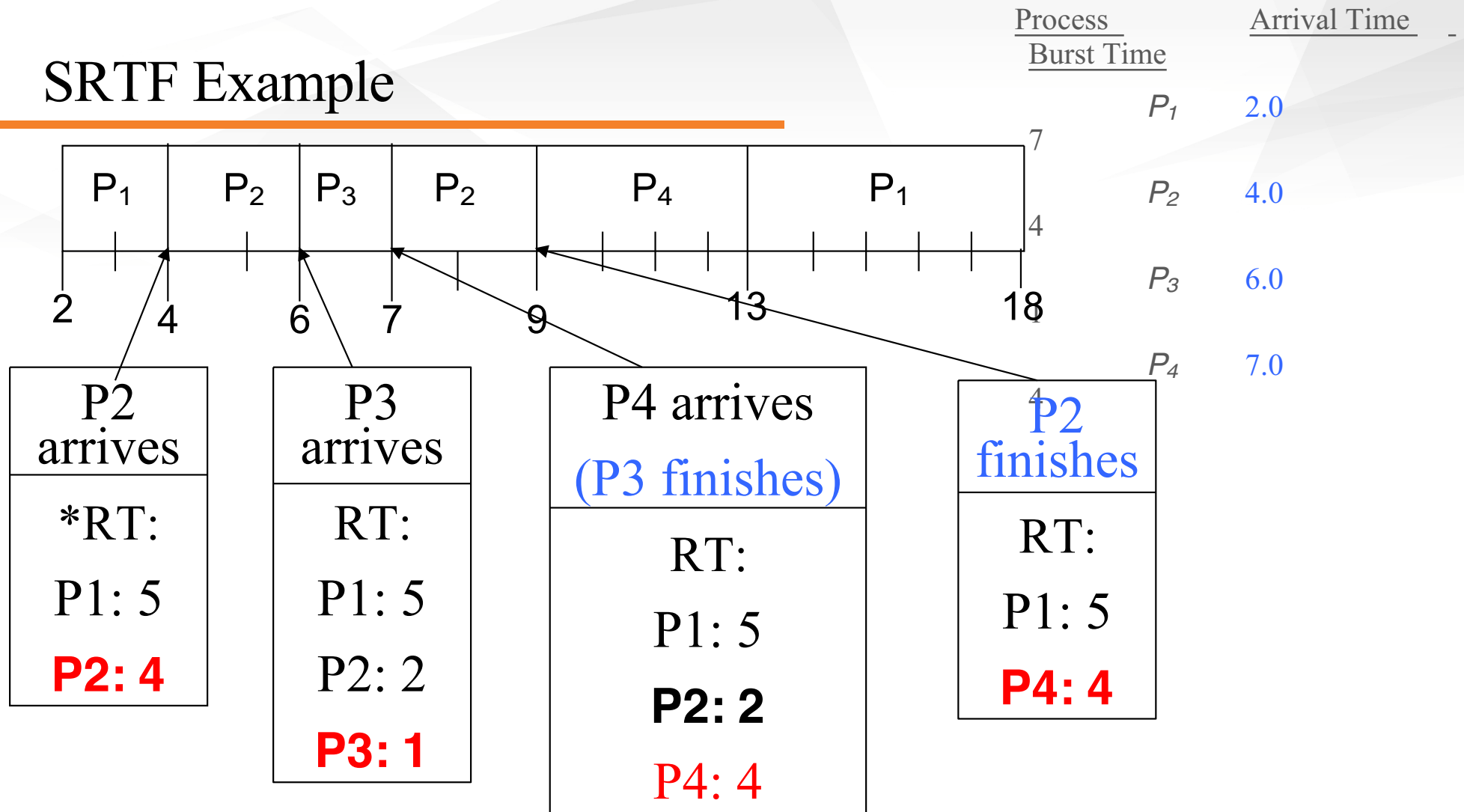
$$\text{Average Turnaround Time} = \text{ATT} = (7 + 10 + 4 + 11)/4 = 32/4 = 8$$

Waiting time (Time spent in ready queue)

$$T_{w,p1} = 0; T_{w,p2} = 6; T_{w,p3} = 3; T_{w,p4} = 7$$

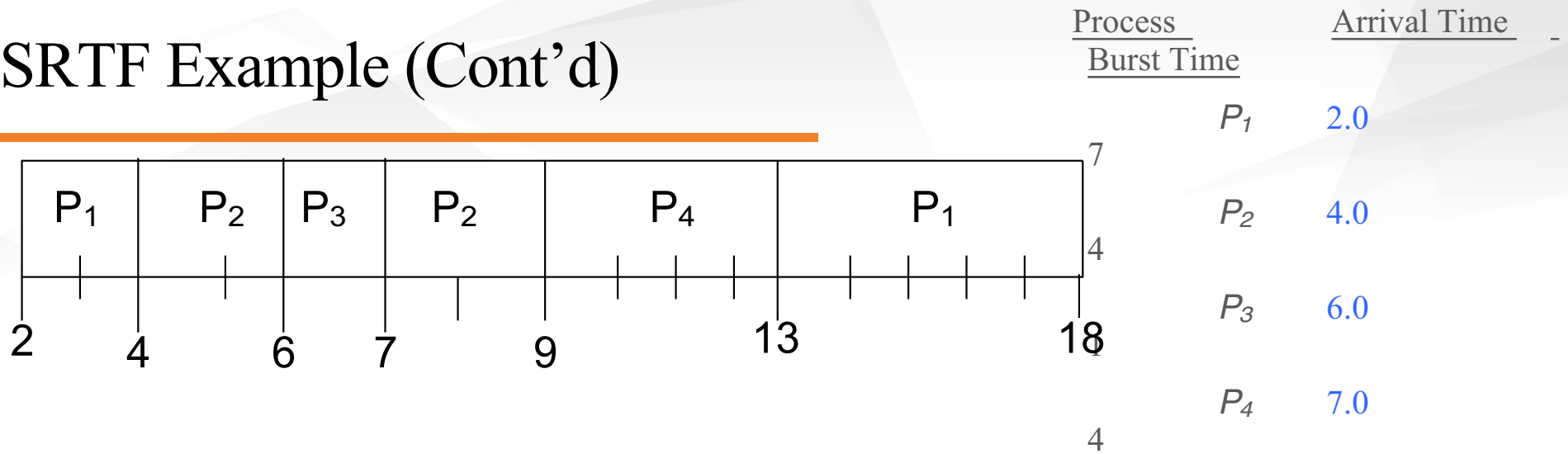
$$\text{Average Waiting Time} = \text{AWT} = (0 + 6 + 3 + 7)/4 = 16/4 = 4$$

SRTF Example



*RT: Remaining Time

SRTF Example (Cont'd)



Response Time: (Time First Time On CPU – Arrival Time)

$$T_{r,p1} = (2 - 2) = 0; T_{r,p2} = (4 - 4) = 0; T_{r,p3} = (6 - 6) = 0; T_{r,p4} = (9 - 7) = 2$$

$$\text{Average Response Time} = \text{ART} = (0 + 0 + 0 + 2)/4 = 2/4 = 0.5$$

Turnaround time (Completion Time – Arrival Time)

$$T_{T,p1} = (18 - 2) = 16; T_{T,p2} = (9 - 4) = 5; T_{T,p3} = (7 - 6) = 1; T_{T,p4} = (13 - 7) = 6$$

$$\text{Average Turnaround Time} = \text{ATT} = (16 + 5 + 1 + 6)/4 = 28/4 = 7$$

Waiting time (Time spent in ready queue)

$$T_{w,p1} = 9; T_{w,p2} = 1; T_{w,p3} = 0; T_{w,p4} = 2$$

$$\text{Average Waiting Time} = \text{AWT} = (9 + 1 + 0 + 2)/4 = 12/4 = 3$$